

מבני נתונים - תרגיל מעשי 2 – ערימת פיבונאצ'י

ענבל ברזילי, שם משתמש - inbalb4, ת.ז. - 212321202
שרון שוורצמן, שם משתמש - shwartzman1, ת.ז. - 206479081

מקבלת: כלום מחזירה: מופע חדש של <code>FibonacciHeap</code> מאתחלת ערימת פיבונאצ'י ריקה עם השדות <code>min</code>, <code>rootsListHead</code>, <code>totalLinks</code>, <code>totalCuts</code>, <code>size</code> ו-<code>numTrees</code> מאופסים לערכי ברירת מחדל. מאתחלת את כל השדות ל- <code>0</code> / <code>null</code> ומגדירה <code>shouldConsolidate</code> לערך <code>true</code>. סיבוכיות: $O(1)$	<code>FibonacciHeap()</code>
מקבלת: מפתח (מספר שלם) ומידע (מחרוזת) מחזירה: ה-<code>HeapNode</code> החדש שנוצר יוצרת <code>HeapNode</code> חדש עם המפתח והמידע שהתקבלו, מוסיפה אותו לרשימת השורשים, מעדכנת את מצביע המינימום אם צריך, מגדילה את <code>size</code> ו-<code>numTrees</code>. סיבוכיות: $O(1)$ - הפעולה משרשרת את ה- <code>HeapNode</code> החדש לתחילת רשימת השורשים בשינוי מצביעים.	<code>insert(int key, String info)</code>
מקבלת: כלום מחזירה: <code>HeapNode</code> עם המפתח המינימלי, או <code>null</code> אם הערימה ריקה. מחזירה את שדה <code>min</code> שמתוחזק לאורך כל הפעולות. סיבוכיות: $O(1)$	<code>findMin()</code>
מקבלת: כלום מחזירה: כלום מסירה את צומת המינימום, מוסיפה את ילדיו לרשימת השורשים, מבצעת <code>successiveLinking</code> לאיחוד עצים אם <code>shouldConsolidate</code> הוא <code>true</code>, מעדכנת את מצביע המינימום ומקטינה את <code>size</code>. סיבוכיות: $O(\log n)$ amortized - הפעולה מסירה את המינימום מהערימה וקוראת ל- <code>successiveLinking</code>, פירוט הסיבוכיות בפעולה זו.	<code>deleteMin()</code>
מקבלת: צומת <code>HeapNode x</code> וערך הפרש <code>diff</code>. מחזירה: כלום מקטינה את המפתח של צומת <code>x</code> ב-<code>diff</code>, ואם המפתח החדש קטן ממפתח ההורה מבצעת <code>cascadingCut</code> לשמירה על תכונות הערימה. סיבוכיות: $O(1)$ amortized - הפעולה מקטינה את ערך המפתח של <code>x</code> וקוראת ל- <code>cascadingCut</code> על מנת לתקן את הערימה, פירוט הסיבוכיות בפעולה זו.	<code>decreaseKey(HeapNode x, int diff)</code>
מקבלת: צומת <code>HeapNode x</code> למחיקה. מחזירה: כלום מקטינה את מפתח הצומת ל- 0 באמצעות <code>decreaseKey</code>, ואז מסירה אותו באמצעות <code>deleteMin</code>. מגדירה את <code>shouldConsolidate</code> להיות <code>false</code> לפני הקריאה ל- <code>deleteMin</code> על מנת שלא נבצע <code>successiveLinking</code>, אלא אם מוחקים את המינימום. מחזירה את ערכו ל- <code>true</code> לאחר מכן. סיבוכיות: $O(\log n)$ amortized - הפעולה קוראת ל- <code>decreaseKey</code> ולאחר מכן ל- <code>deleteMin</code>, המשתמשות בפעולות <code>successiveLinking</code>, <code>cascadingCut</code>, פירוט הסיבוכיות בפעולות אלה.	<code>delete(HeapNode x)</code>

מקבלת: מופע נוסף של FibonacciHeap. מחזירה: כלום. ממזגת את heap2 לתוך הערימה הנוכחית. משרשרת את רשימות השורשים של שתי הערימות, מעדכנת את totalLinks הסטטיסטיקה ושות' totalCuts. 1 - סיבוכיות: $O(1)$ - הפעולה משרשרת את רשימות השורשים של הערימה הנוכחית ושל heap2 בשינוי מצביעים.	meld(FibonacciHeap heap2)
קבלת: כלום. מחזירה: מספר האיברים הנוכחי (מספר שלם). מחזירה את גודל הערימה השמור בשדה size המתוחזק. 1 - סיבוכיות: $O(1)$	size()
מקבלת: כלום. מחזירה: מספר העצים ברשימת השורשים (מספר שלם). מחזירה את מספר העצים השמור בשדה numTrees המתוחזק. 1 - סיבוכיות: $O(1)$	numTrees()
מקבלת: כלום. מחזירה: מספר כולל של פעולות חיבור עצים (מספר שלם). מחזירה את המספר הכולל של פעמים שעצים חוברו השמור במונה totalLinks המתוחזק. 1 - סיבוכיות: $O(1)$	totalLinks()
מקבלת: כלום. מחזירה: מספר כולל של פעולות חיתוך (מספר שלם). מחזירה את המספר הכולל של פעמים שצמתים נותקו מההורה השמור במונה totalCuts המתוחזק. 1 - סיבוכיות: $O(1)$	totalCuts()
מקבלת: שני צמתי HeapNode לקישור. מחזירה: צומת ההורה המתקבל. פונקציית עזר private שמקשרת שני עצים על ידי הפיכת אחד הצמתים לבן של השני. משווה מפתחות, הופכת את הצומת עם המפתח הגדול יותר לבן של הצומת עם המפתח הקטן יותר, מעדכנת את דרגת ההורה ואת totalLinks. 1 - סיבוכיות: $O(1)$ - הפעולה מחברת בין שני עצים בערימה בשינוי מצביעים.	link(HeapNode node1, HeapNode node2)
מקבלת: צומת HeapNode לחיתוך. מחזירה: כלום. פונקציית עזר private שחותכת צומת מההורה שלו. מסירה את הצומת מרשימת הילדים של ההורה, מוסיפה לרשימת השורשים, מעדכנת דרגות 1 - marked, מגדילה את totalCuts. 1 - סיבוכיות: $O(1)$ - הפעולה מנתקת צומת מההורה שלו ומשרשרת אותו לרשימת השורשים בשינוי מצביעים.	cut(HeapNode node)

מקבלת: צומת HeapNode שבו מתחיל cascading cut. מחזירה: כלום פונקציית עזר private שמבצעת פעולת cascading cut. חותכת את הצומת מההורה, אם ההורה מסומן ואינו שורש, חותכת גם את ההורה מההורה שלו באופן רקורסיבי. סיבוכיות: $O(1)$ amortized - הפעולה נקראת אך ורק מהפעולות decreaseKey, delete, וכן כל צומת מאבד בן אחד לכל היותר כל עוד הוא בן של צומת אחר. לכן, משיטת הבנק, אם נשים 2 מטבעות על כל צומת שהוא marked, מטבע אחד יממן את פעולת סימון הצומת והשני יממן את החיתוך העתידי של הצומת מההורה שלו. לכן הסיבוכיות היא $O(1)$ amortized.	cascadingCut(HeapNode node)
מקבלת: כלום מחזירה: כלום פונקציית עזר private שמאחדת עצים בערימה על ידי קישור שורשים מאותה דרגה. משתמשת במערך המכיל את שורשי הערימה לפי דרגה למציאת וחיבור עצים מאותה דרגה עד שאין שני שורשים מאותה דרגה. סיבוכיות: $O(\log n)$ amortized - פעולת link מפחיתה ב - 1 את מספר העצים בערימה. לכן, משיטת הבנק, אם נשים 2 מטבעות על כל צומת שנכנס למבנה, מטבע אחד יממן את פעולת ההכנסה והשני יממן את פעולת החיבור בין עצים העתידית. לאחר הפעולה יהיו בערימה $O(\log n)$ עצים ולכן הסיבוכיות היא $O(\log n)$ amortized.	successiveLinking()
מקבלת: מפתח (מספר שלם) ומידע (מחרוזת) מחזירה: מופע חדש של HeapNode מאתחלת צומת ריק בערימת פיבונאצ'י עם השדות key, info, child, next, prev, parent, rank מאופסים לערכי ברירת מחדל. מגדירה את key ואת info להיות ערכי הפרמטרים שמתקבלים ואת שאר השדות לערכי ל - 0 / null. סיבוכיות: $O(1)$	HeapNode(int key, String info)
מקבלת: צומת HeapNode שיהיה בן חדש של הצומת הנוכחי. מחזירה: כלום פונקציית עזר private במחלקה HeapNode שמוסיפה לצומת הנוכחי בן חדש. מוסיפה את הבן החדש לרשימת הבנים של הצומת הנוכחי, מעדכנת את דרגת הצומת הנוכחי. סיבוכיות: $O(1)$ - הפעולה מוסיפה בן חדש לצומת בערימה בשינוי מצביעים.	addChild(HeapNode newChild)

מספר סידורי i	זמן ריצה (מילישניות)	גודל הערימה בסיום	מספר חיבורים	מספר חיתוכים	מספר עצים בסיום
1	0.75	6559.0	6550.0	0.0	9.0
2	0.95	19681.0	19674.0	0.0	7.0
3	3.9	59047.0	59037.0	0.0	10.0
4	9.9	177145.0	177133.0	0.0	12.0
5	39.75	531439.0	531427.0	0.0	12.0

מבצעים n פעולות הכנסה, כל אחת בסיבוכיות $O(1)$, שבסופן מתקבלת ערימה עם n עצים בדרגה 0. לאחר מכן, מוחקים את המינימום פעם אחת ובמהלך פעולה זו מתבצעת הפעולה successive linking בסיבוכיות $O(n)$ (מעבר על כל $n - 1$ העצים וחיבור עצים מאותה דרגה עד שכל העצים הם מדרגה שונה). סך הכל נקבל סיבוכיות $O(n)$.

סדר ההכנסה אינו משפיע על המדידות. בהכנסה נקבל לכל סדר הכנסה n עצים מדרגה 0. מכיוון שמוחקים את המינימום פעם אחת בלבד, והמינימום בהכרח עץ מדרגה 0, לא מתבצעים חיתוכים ולכן בהכרח נקבל $n - 1$ עצים מדרגה 0. כלומר, תמיד נבצע successive linking על ערימה עם $n - 1$ עצים מדרגה 0 ונקבל בסוף ערימה בעלת מבנה עצים זהה בכל פעם.

ניסוי שני:

מספר סידורי i	זמן ריצה (מילישניות)	גודל הערימה בסיום	מספר חיבורים	מספר חיתוכים	מספר עצים בסיום
1	3.15	3280.0	39828.1	36553.1	5.0
2	8.8	9841.0	135250.85	125416.85	7.0
3	31.2	29524.0	450844.65	421328.65	8.0
4	95.4	88573.0	1498982.55	1410421.55	12.0
5	361.65	265720.0	4887989.7	4622278.7	9.0

מבצעים n פעולות הכנסה, כל אחת בסיבוכיות $O(1)$, שבסופן מתקבלת ערימה עם n עצים בדרגה 0. לאחר מכן מוחקים את המינימום $\frac{n}{2}$ פעמים. המחיקה הראשונה מתבצעת בסיבוכיות $O(n)$ (כמתואר עבור הניסוי הראשון). בסוף המחיקה, נקבל ערימה ובה $O(\log n)$ עצים. לאחר מכן, עבור ערימה בעלת m איברים, נקבל לכל מחיקה סיבוכיות $O(\log m)$. כלומר, הסיבוכיות הכוללת המתקבלת היא

$$\begin{aligned}
 O(n) + \sum_{i=\lceil \frac{n}{2} \rceil}^{n-1} \log(i) &= O(n) + \left(\log\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + \log\left(\left\lfloor \frac{n}{2} \right\rfloor + 1\right) + \dots + \log(n-1) \right) \leq \\
 &\leq O(n) + O\left(\frac{n}{2}\right) \log(n-1) = O(n) + O(n) O(\log n) = O(n \log n) \\
 O(n) + \sum_{i=\lceil \frac{n}{2} \rceil}^{n-1} \log(i) &= O(n) + \left(\log\left(\left\lfloor \frac{n}{2} \right\rfloor\right) + \log\left(\left\lfloor \frac{n}{2} \right\rfloor + 1\right) + \dots + \log(n-1) \right) \geq \\
 &\geq O(n) + O\left(\frac{n}{2}\right) \log\left(\left\lfloor \frac{n}{2} \right\rfloor\right) = O(n) + O(n) O(\log n) = O(n \log n)
 \end{aligned}$$

כלומר, הסיבוכיות היא $\Theta(n \log n)$.

סדר ההכנסה משפיע על מדידות מספר החיבורים ומספר החיתוכים, וכתוצאה מכך גם על זמן הריצה. נשים לב, כי אם המינימום בערימה הוא שורש של עץ מדרגה גבוהה, אז נבצע חיתוכים למספר בנים רב של המינימום. כתוצאה מכך, נקבל כי מספר העצים לאחר מחיקת המינימום גדל כתלות במספר הבנים. ככל שמספר העצים רב יותר, מספר החיבורים שייתבצעו בזמן successive linking גדל. בנוסף, ככל שנבצע יותר חיבורים ויותר חיתוכים נקבל כי זמן הריצה ארוך יותר.

אופן המימוש משפיע על סדר העצים והצמתים בערימה לאחר successive linking, אך אם נקבל שהמינימום הוא שורש של עץ מדרגה גבוהה מספר רב של פעמים בזמן הניסוי, מספר החיבורים והחיתוכים יגדל בהתאם וכך זמן הריצה.

ניסוי שלישי:

מספר סידורי i	זמן ריצה (מילישניות)	גודל הערימה בסיום	מספר חיבורים	מספר חיתוכים	מספר עצים בסיום
1	0.35	31.0	6550.0	6549.85	30.85
2	3.15	31.0	19674.0	19673.95	30.95
3	6.2	31.0	59037.0	59037.0	31.0
4	18.9	31.0	177133.0	177132.95	30.95
5	96.65	31.0	531427.0	531426.95	30.95

מבצעים n פעולות הכנסה, כל אחת בסיבוכיות $O(1)$, שבסופן מתקבלת ערימה עם n עצים בדרגה 0. לאחר מכן, מוחקים את המינימום פעם אחת ובמהלך פעולה זו מתבצעת הפעולה successive linking בסיבוכיות $O(n)$. בנוסף, מבצעים $(2^5 - 1) - n$ מחיקות של המקסימום בערימה (גישה למקסימום מתבצעת בעזרת מערך של הצמתים בערימה ולכן עולה $O(1)$). בפעולת המחיקה, מורידים את ערכו של המפתח של המקסימום ל-0 באמצעות decrease key בסיבוכיות $O(1)$ amortized. במהלך הפעולה מנתקים את המקסימום מההורה שלו ובמידה וההורה מסומן ממשיכים במעלה העץ ומפעילים את פעולת החיתוך רקורסיבית.

המקסימום בערימה הוא בהכרח עלה, ולכן אין צורך לבצע פעולת meld על מנת להוסיף את בניו לערימה לאחר המחיקה. בנוסף, בעת המחיקה של צומת שאינו המינימום לא מבצעים successive linking. לכן, הסיבוכיות הכוללת ניתנת לחישוב על ידי חיבור $(2^5 - 1) - n$ פעולות decrease key בסיבוכיות $O(1)$ amortized, n פעולות הכנסה בסיבוכיות $O(1)$ כל אחת ומחיקת מינימום אחת בסיבוכיות $O(n)$. נקבל סך הכל

$$(n - (2^5 - 1)) \cdot O(1) \text{ amort.} + n \cdot O(1) + O(n) = O(n) + O(n) + O(n) = O(n)$$

סדר ההכנסה משפיע על מספר החיתוכים ועל מספר העצים בסיום הניסוי. מספר החיבורים לא משתנה משום שמבצעים successive linking פעם אחת בלבד במחיקת המינימום ותמיד על $1 - n$ עצים מדרגה 0.

נשים לב כי בסיום תמיד יישארו הצמתים בעלי המפתחות בטווח $2 - 32$ ומכיוון שלא מבצעים successive linking במחיקת המקסימום, מצב הצמתים לאחר מחיקת המינימום וביצוע successive linking נשאר דומה. כלומר, סיכוי גבוה ששני צמתים באותו עץ יישארו באותו עץ וששני צמתים בעצים שונים יישארו בעצים שונים. בפרט, מכיוון שכמות הצמתים והצמתים שנשארים הם מינימליים (כלומר ברמה גבוהה בערימה) הסיכוי שאחד הצמתים האלו יהיה מסומן ויחתך מהעץ נמוך. מספר העצים מושפע ממצב הצמתים לאחר successive linking ולכן עלול להשתנות כתלות בסדר ההכנסה ובמימוש.

מכיוון שמקסימום עלה, מספר החיתוכים מושפע אך ורק מפעולת cascading cut ותלוי במספר הצמתים המסומנים בעץ שבו נמצא המקסימום. ככל שיש יותר צמתים מסומנים, ייתבצעו יותר חיתוכים כיוון שנעלה בעץ לכל צומת מסומן.

נסמן:

links – מספר החיבורים הכולל

cuts – מספר החיתוכים הכולל

size – גודל הערימה בסיום הניסוי

trees – מספר העצים בערימה בסיום הניסוי

נקבל את המשוואה הבאה המתארת את הקשר בין ערכים אלו $trees = size - links + cuts$.

אם לא מבצעים חיתוכים או חיבורים כלל, מספר העצים בערימה שווה לגודל הערימה בסיום (כל העצים נותרים בדרגה 0).

מבצעים פעולת חיבור עצים אך ורק בפעולת successive linking, פעולת חיתוך מתבצעת במחיקת המינימום או בפעולת decrease key. בפעולת חיבור עצים, מספר העצים בערימה יורד ב-1. בפעולת חיתוך בין צמתים מספר העצים בערימה גדל ב-1.

כלומר, נרצה להחסיר מגודל הערימה את מספר החיבורים (כי מספר העצים יורד בפעולת חיבור) ולהוסיף את מספר החיתוכים (כי מספר העצים גדל בפעולת חיתוך). לכן נקבל את המשוואה המתוארת.